排序算法记忆树

> 本质：不同排序算法，本质上都是在想办法"把小的放前面，大的放后面"。只是"怎么找、怎么换"的方式不同。

排序算法

- 冒泡排序
- 核心：从左往右，**相邻比较**，不符合就交换
- 特征：
- 一轮后最大的“冒泡”到末尾 → 末尾已排序
- 下一轮无需再比末尾元素
- 某一轮无交换 → 提前终止（优化点）
- 时间复杂度：平均/最坏 $O(n^2)$，最好 $O(n)$
- 空间复杂度：$O(1)$
- 适用场景：教学示例、数据量小、需要简单实现的场景
- 一句话：**一路比，一路换**
- 选择排序
- 核心：每轮找**最小/最大值**，放到未排序区域首位
- 特征：
- 一轮只搞定一个元素
- 无论如何都要扫一遍找最值
- 剩余区域大小线性缩小：n → n-1 → n-2 ...
- 时间复杂度：**永远 $O(n^2)$**（无最好情况）
- 空间复杂度：$O(1)$
- 适用场景：数据量小、对实现简单性要求高、无需考虑效率的场景
- 一句话：**先找，再换**
- 【冒泡 vs 选择】
- 冒泡：一路比较，一路换（边找边换）
- 选择：先全部扫一遍找最值，再一次性交换
- 剩余区域：两者都是每次缩 1 个（线性缩小）

- 插入排序
- 核心：从右往左，逐个和前面**已排序**元素比较，找到位置就插入
- 特征：
- 前面始终是有序的，后面是无序的
- 像打牌时整理手牌
- 对已有序的数组特别快
- 时间复杂度：平均/最坏 $O(n^2)$，最好 $O(n)$
- 空间复杂度：$O(1)$
- 适用场景：数据量小、基本有序的数组、在线插入排序场景
- 一句话：**拿一个，插进去**
- 希尔排序（插入排序的改进版）
- 核心：**分组插入排序** — 先远距离比较交换，再逐步缩小间隔
- 特征：
- gap 序列从大到小（常用 n/2 每次减半）
- 最后一轮 gap=1 就是标准插入排序
- 大间隔让元素快速归位，小间隔时数组已接近有序
- 代码结构和插入排序几乎一样，只是把 1 换成 gap
- 时间复杂度：约 $O(n^{\wedge}1.3)$，比 $O(n^2)$ 好
- 空间复杂度：$O(1)$
- 稳定性：❌ 不稳定
- 适用场景：中等规模数据、对插入排序做优化时使用
- 一句话：**大步跳，小步走**
- 快速排序 💥
- 核心：选**基准(pivot)**，分区成 [小 | 基准 | 大]，递归排序左右
- 特征：
- 分治思想（Divide and Conquer）
- 原地版：Lomuto / Hoare 分区
- 非原地版：创建 left / right / equal 三个数组
- 剩余区域：**指数级缩小** n → n/2 → n/4 ...
- 时间复杂度：
- 平均 $O(n \log n)$
- 最好 $O(n \log n)$（基准在中间）

- 最坏 O(n²)（数组已有序 + 选极端基准）
- 空间复杂度：O(log n)（递归栈）
- 稳定性：❌ 不稳定
- 适用场景：大多数实际项目中的默认排序选择，适合大规模数据
- 一句话：**一刀切成两半，各切各的**
- 归并排序
- 核心：**拆 → 排 → 合**
- 特征：
- 先递归拆到最小单元（每个 1 个元素）
- 然后两两合并，合并时排序
- 稳定排序（相同元素相对位置不变）
- 需要额外空间存临时数组
- 时间复杂度：**永远 O(n log n)**
- 空间复杂度：O(n)
- 稳定性：✅ 稳定
- 适用场景：需要稳定排序、外部排序、归并多个有序段
- 一句话：**先拆碎，再拼好**
- 堆排序
- 核心：建**大顶堆**，每次把堆顶（最大值）拿出来放末尾
- 特征：
- 利用完全二叉树结构
- 堆顶永远是当前最大值
- 原地排序
- 时间复杂度：**永远 O(n log n)**
- 空间复杂度：O(1)
- 稳定性：❌ 不稳定
- 适用场景：需要原地排序、优先队列、选择前 K 大元素
- 一句话：**每次拿最大的出来**

复杂度对比速查表

算法	最好	平均	最坏	空间	稳定
冒泡	O(n)	O(n²)	O(n²)	O(1)	✅
选择	O(n²)	O(n²)	O(n²)	O(1)	❌
插入	O(n)	O(n²)	O(n²)	O(1)	✅
希尔	O(n)	O(n^1.3)	O(n²)	O(1)	❌
快排	O(n log n)	O(n log n)	O(n²)	O(log n)	❌
归并	O(n log n)	O(n log n)	O(n log n)	O(n)	✅
堆排	O(n log n)	O(n log n)	O(n log n)	O(1)	❌

记忆口诀

算法	口诀	核心区别
冒泡	一路比，一路换	相邻交换，最大的先冒泡
选择	先找，再换	每轮只换一次，线性缩小
插入	拿一个，插进去	前面有序，拿后面的往前面插
希尔	大步跳，小步走	分组插排，gap 从大到小
快排	一刀两半各切各的	分治，指数级缩小
归并	先拆碎再拼好	分治，稳定，需额外空间
堆排	每次拿最大的	完全二叉树，原地，不稳定

稳定性记忆技巧

- ✅ **稳定**：冒泡、插入、归并 —— "冒插归" → 谐音"**冒茶归**"（冒着茶气归来）
- ❌ **不稳定**：选择、希尔、快排、堆排 —— 剩下的全是不稳定的

第二部分：复杂度与稳定性专题

> 核心思想：**时间复杂度看"做事要花多少功夫"；空间复杂度看"做事要占多少额外空间"**

一、时间复杂度

本质

不是程序运行了多少秒，而是 **当数据越来越大时，算法执行次数增长得有多快**。

⚠ 重点：看**增长趋势**，不看绝对时间。

常见时间复杂度一览（从小到大）

> 增长速度：$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

复杂度	含义	核心特征	经典例子
$O(1)$	数据再多，操作次数基本不变	固定次数	数组按下标访问、哈希表查找
$O(\log n)$	每次砍一半	规模快速缩小	二分查找、快速排序平均
$O(n)$	每个数据走一遍	线性扫描	顺序查找
$O(n \log n)$	分组处理 + 合并	分治思想	归并排序、快速排序平均
$O(n^2)$	数据之间反复比较	双重循环	冒泡、选择、插入的平均/最坏
$O(2^n)$	每多一个元素，工作量翻倍	指数爆炸	斐波那契递归（最朴素版）
$O(n!)$	所有排列组合一遍	阶乘爆炸	旅行商问题暴力枚举

记忆口诀

- $O(1)$ → “直接拿”
- $O(\log n)$ → “砍一半”
- $O(n)$ → “走一遍”
- $O(n \log n)$ → “分着来”
- $O(n^2)$ → “两两比”
- $O(2^n)$ → “翻一倍”
- $O(n!)$ → “全排列”

二、空间复杂度

本质

算法**额外占了多少内存**。

⚠️ 重点：关注**额外辅助空间**，原本存储输入数据的空间不算！

> 类比：你家厨房本来就要装冰箱（输入数据），空间复杂度关心的是你**额外买了几个锅、几个碗**。

常见空间复杂度

复杂度	含义	特征	例子
O(1)	额外空间基本固定	不随数据量增长	冒泡、选择、插入、希尔、堆排
O(log n)	随 log n 增长	递归调用栈	快速排序（原地版递归栈）
O(n)	随数据量线性增长	和数据量一样	归并排序（临时合并数组）、快排非原地版

容易误解的点

误解	正确理解
❌ O(1) 表示完全不占内存	✅ O(1) 表示额外空间**固定**，不随数据量增长。比如交换需要一个临时变量，这个变量永远只占 1 个位置
❌ 空间复杂度低 = 算法更好	✅ 要看场景！比如归并排序 O(n) 空间但时间**永远** O(n log n)；快排平均也快但最坏 $O(n^2)$

三、空间复杂度的意义

为什么要看空间复杂度?

1. 内存够不够？ → 数据量很大时，额外内存可能成为瓶颈 → 比如归并排序 O(n) 空间，1亿条数据就要额外 400MB+ 内存
2. 速度与空间如何取舍？ → 归并排序：用 O(n) 空间换"永远 O(n log n)"的稳定时间 → 快排原地版：用 O(log n) 空间换平均 O(n log n)，但最坏退化

3. 算法是否适合当前场景？ → 嵌入式设备/手机：内存有限，优先 O(1) 空间的算法 → 大数据处理：允许 O(n) 空间换稳定速度

四、稳定性

定义

相同关键字的元素，排序后相对顺序是否改变？

类型	含义
✅ 稳定	相同元素的原有相对顺序**不会**改变
❌ 不稳定	相同元素的原有相对顺序**可能**改变（不是一定会变！）

例子说明

假设数组 `[2(红), 2(蓝), 1]`，前面的数字是值，括号是颜色用来区分相同值：
稳定排序结果：`[1, 2(红), 2(蓝)]` ← 红 2 原本在蓝 2 前面，排序后还在前面
不稳定排序结果：`[1, 2(蓝), 2(红)]` ← 红 2 和蓝 2 顺序可能交换

记忆技巧（必背！）

类型	算法	记忆口诀
✅ **稳定**	冒泡、插入、归并	**"冒插归"** → 谐音**"冒着茶气归来"**
❌ **不稳定**	选择、希尔、快排、堆排	剩下的全是不稳定

第三部分：速查表 & 记忆点

一、排序算法三大维度终极对比

算法	最好时间	平均时间	最坏时间	空间	稳定	一句话
冒泡	O(n)	$O(n^2)$	$O(n^2)$	O(1)	✅	一路比一路换
选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	❌	先找再换

算法	最好时间	平均时间	最坏时间	空间	稳定	一句话
插入	O(n)	O(n²)	O(n²)	O(1)	✅	拿一个插进去
希尔	O(n)	O(n^1.3)	O(n²)	O(1)	❌	大步跳小步走
快排	O(n log n)	O(n log n)	O(n²)	O(log n)	❌	一刀两半各切各的
归并	O(n log n)	O(n log n)	O(n log n)	O(n)	✅	先拆碎再拼好
堆排	O(n log n)	O(n log n)	O(n log n)	O(1)	❌	每次拿最大的

二、记忆口诀汇总

类别	口诀
稳定排序	**冒插归**（冒着茶气归来）
原地排序（O(1)空间）	冒选插希堆（冒泡、选择、插入、希尔、堆排）
永远 O(n log n) 时间	归并、堆排
平均最快	快排

三、核心对比结论

- 时间复杂度最优（永远 O(n log n)）：归并、堆排
- 空间复杂度最优（O(1)）：冒泡、选择、插入、希尔、堆排
- 最稳定的组合（时间+空间+稳定）：归并（但空间 O(n) 是硬伤）
- 最均衡的算法：快排（平均最快，但最坏退化）

四、复杂度记忆点

时间复杂度

1. 看增长趋势，不看绝对秒数
2. 分治思想 = O(n log n)（归并、快排平均）
3. 双重循环 = O(n²)（冒泡、选择、插入）

4. 每次砍一半 = O(log n)（二分）
5. 递归指数级 = O(2^n)（朴素斐波那契）

空间复杂度

---

1. O(1)：额外空间基本固定，不随数据量增长
2. O(n)：额外空间和数据量线性增长
3. 看的是增长趋势，不是某一刻的字节数
4. 原地排序基本都是 O(1)（快排除外有 log n 的递归栈）
5. 空间换时间是常见取舍（归并、计数排序、基数排序）
